**UNIVERSITY OF ALGARVE**

Faculty of Sciences and Technology

Department of Electronic Engineering and Informatics

# Laboratory Tutorial 1

## Small Instances of the 8-Puzzle

*Applying the Strategy design pattern to state-space search*

Course: Artificial Intelligence

Academic year: 2026/2027

Instructors: Gonçalo Serafim (gfserafim@ualg.pt) and José Valente de Oliveira (jvo@ualg.pt)

Disclaimer: This tutorial was adapted from the original with the assistance of GPT-6 Astra to improve its formatting, visual clarity and writing quality.

**Group work and submission.** Complete this tutorial as a group and submit your solution using your group's account on Mooshak by **28 September 2026**. Select **Contest Artificial Intelligence 2026-27**, then **Problem A — Lab Tutorial 1**.

## Contents

**1 Create your Mooshak account** **2**

**2 The 8-puzzle** **2**

**3 Your task** **2**

3.1 What is Strategy, and why use it? . . . 2

**4 How the search works** **3**

4.1 Expanding a state: an example . . . 3

**5 Program design and setup** **4**

5.1 The Strategy Class Diagram . . . 4

5.2 The Ilayout interface . . . 4

5.3 Set up the project in VS Code . . . 4

**6 Implement and test the board** **5**

6.1 Implement toString() and pass its tests . . . 5

6.2 Complete the remaining Board methods . . . 7

**7 Implement the search algorithm** **7**

**A Small 8-Puzzle Instances** **9**

A.1 Sample 1 . . . 9

A.2 Sample 2 . . . 9

**B JUnit Assertions** **10**

# 1 Create your Mooshak account

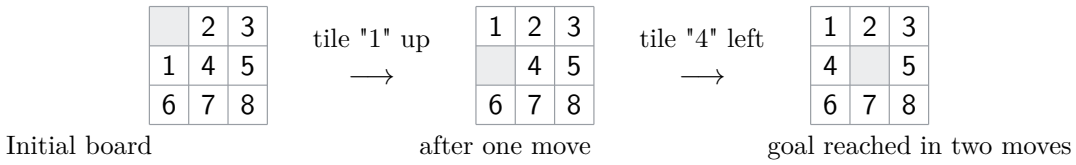
1. Choose your lab class and join a group on the course registration page.
2. Open Mooshak and select **Register [for On-line Contest]**. Enter the following details:

| | |
|---|---|
| **Contest** | Artificial Intelligence 2026-27 |
| **Group** | Your lab class (PL1,PL2,PL3,...) |
| **Name** | Write: "IA2627P<i>G<j>", where <i> is your lab class number (1,2,3,4 or 5) and <j> is your group number, as stated in the course registration page. For example, "IA2627P3G1" |
| **Email** | One group member's email address, which will receive the group's password |

3. Log in and select **Problem A – Lab Tutorial 1**. Click **View** to read the problem statement. Clicking **Ask** allows you to send a question to the instructor. Clicking **Choose File** opens a file browser so that you can choose the work you wish to submit; you can also drag and drop what you wish to submit to said button rather than pressing it. Finally, **Submit** submits the work selected with "Choose File". After a submission, Mooshak will either accept your work, or display an error. If needed, you can click **Help** to understand an error's meaning. **Help** also provides other information besides the meaning of errors, which may be of interest to you.

# 2 The 8-puzzle

The 8-puzzle consists of a $3 \times 3$ board with eight tiles numbered 1–8 and one empty cell. In each move, slide a tile into the empty cell from directly above, below, left or right. Diagonal moves are not allowed. Each move has a cost of 1, and the goal is to find the **shortest sequence of moves** from the initial board to a provided target board.




In the input, each board is written as a sequence of digits, read row by row, with `0` representing the empty cell. The initial board above is `023145678`. Both the initial board and the goal are given as input. Each puzzle used for assessment has a unique shortest solution of at most 12 moves.

# 3 Your task

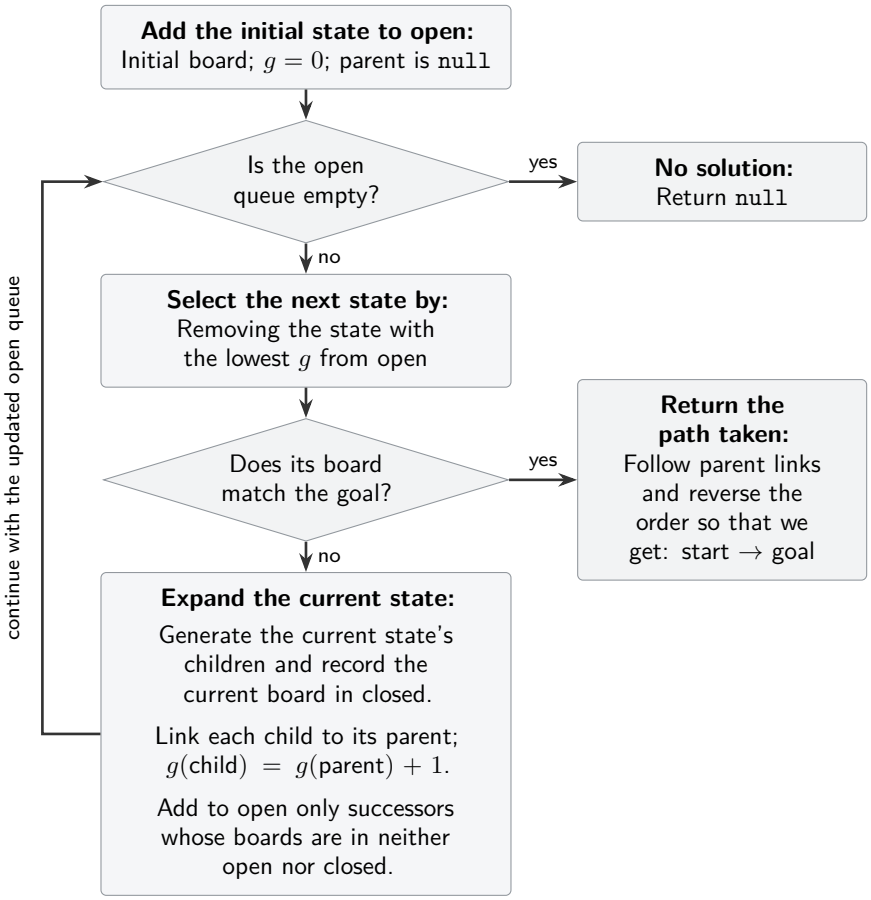
Complete the starter Java project to solve these puzzles using best-first search. Write and run unit tests as you complete the TODOs in `Board` and `BestFirst`. Use the **Strategy design pattern** to keep the puzzle rules separate from the search algorithm. Once your program passes your tests, submit it to Mooshak and check Mooshak's output.

## 3.1 What is Strategy, and why use it?

The Strategy design pattern defines interchangeable behaviour through a shared interface. Here, `BestFirst` uses `Ilayout` to generate legal moves, check whether the goal has been reached and calculate move costs. `Board` implements these operations for the 8-puzzle. To apply the search algorithm to another problem, write a new implementation of `Ilayout`.

# 4 How the search works

A **configuration** describes the positions of the tiles. A search **state** stores a configuration and a link to its parent, as well as the total cost $g$ of the path from the start. The initial state has no parent, and its cost is zero. The **open queue** holds states waiting to be examined, ordered by increasing $g$, which makes less costly states have priority. The **closed list** records configurations that have already been expanded, which is useful for avoiding repetition of previously done work.

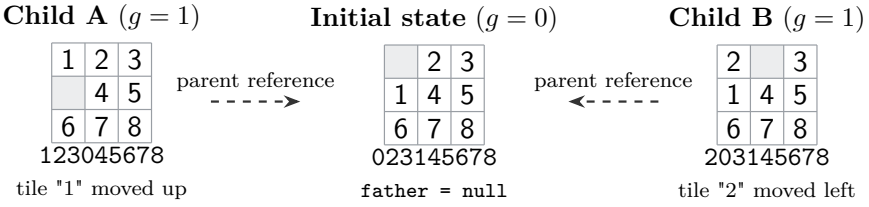



**Note:** This diagram only holds if all moves have the same cost. However, it can easily be adapted to work for the general case.

**Why this approach finds the shortest path:** In this problem, all moves have the same cost (1). Therefore, an algorithm that always expands the node with the least accumulated cost guarantees that the first solution it finds has the lowest possible cost, with said algorithm being a form of **best-first search**. Additionally, a search algorithm whose policy is to expand the node with the least accumulated cost is referred to as **uniform-cost search**.

## 4.1 Expanding a state: an example

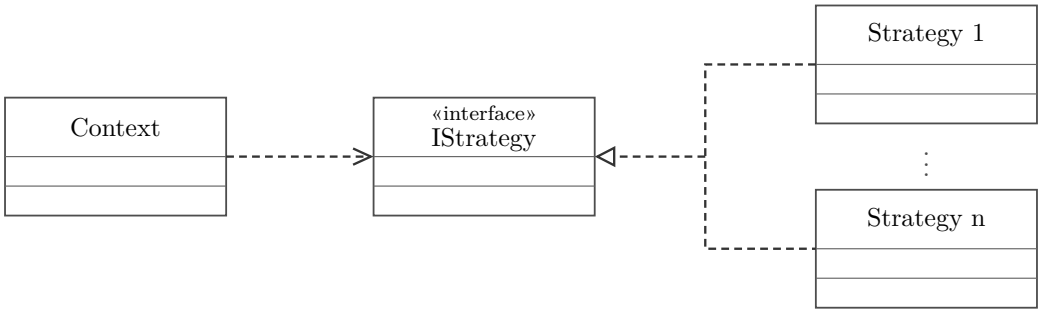
Here, the **initial state** has $g = 0$, as the cost to get from the initial state to itself is 0; its children have $g = 1$, as one move was made and every move in this problem costs 1.




Dashed arrows show each child's stored parent reference (`father`); the parent stores no child references. The search examines states with lower costs first; ties can be broken in any order. Therefore, you could expand either child A or B in the next step.

# 5 Program design and setup

## 5.1 The Strategy Class Diagram




The search uses `Ilayout` as the type of each configuration. For this puzzle, the actual objects are `Boards`: a call to `children()` runs `Board.children()`. The table maps the diagram's roles to the project.

| Pattern role | This project | Responsibility |
|---|---|---|
| Context | `BestFirst` | Order pending states and search for a path. |
| IStrategy | `Ilayout` | Declare legal moves, goal checks and move costs. |
| Strategy 1 | `Board` | Store tiles and implement interface methods. |
| Strategy n | Not Defined | Refers to a potentially different board for another game, for example. |

## 5.2 The Ilayout interface

The supplied `Ilayout` declares three methods; `Board` must implement them:

```
import java.util.List;

interface Ilayout {
    /** Return the configurations reachable in one legal move. */
    List<Ilayout> children();
    /** Return true if this configuration equals l. */
    boolean isGoal(Ilayout l);
    /** Cost of moving into this configuration: 1 for this puzzle. */
    double getK();
}
```

- `children()` returns a list of new configurations, one for each legal move. Each must have its own board storage; the current board must remain unchanged.
- `isGoal(l)` returns `true` when this board has the same tile positions as the goal `l`, and `false` otherwise.
- `getK()` returns the cost of moving into this configuration, from some previous one: always `1.0` in our specific case.

## 5.3 Set up the project in VS Code

1. Install a JDK (version 21 or later), VS Code and Microsoft's Extension Pack for Java, which provides Java editing, running and testing support.
2. Extract `eight-puzzle-starter.zip`. In VS Code, select **File** → **Open Folder** and open the folder containing `src`, `test` and `samples`. Trust the project if prompted.
3. In the **Testing** view (beaker icon), select **Enable Java Tests**, choose **JUnit Jupiter (JUnit 5)**, and allow the library download. If needed, see the VS Code testing guide.

# 6 Implement and test the board

## 6.1 Implement toString() and pass its tests

Open `test/PuzzleUnitTests.java` and run `testConstructor`. You should see a `TODO: toString` failure.

**test/PuzzleUnitTests.java**

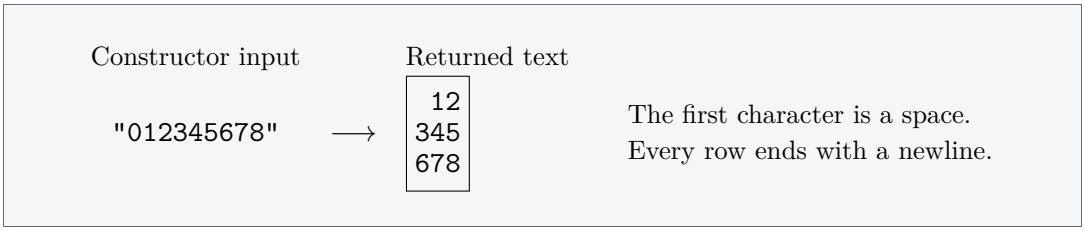
```
import java.io.StringWriter;
import java.io.PrintWriter;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class PuzzleUnitTests {
    @Test
    public void testConstructor() {
        Board b = new Board("023145678");
        StringWriter writer = new StringWriter();
        PrintWriter pw = new PrintWriter(writer);
        pw.println(" 23");
        pw.println("145");
        pw.println("678");
        // JUnit expects the expected value first, then the actual value.
        assertEquals(writer.toString(), b.toString());
        pw.close();
    }

    @Test
    public void testConstructor2() {
        Board b = new Board("123485670");
        StringWriter writer = new StringWriter();
        PrintWriter pw = new PrintWriter(writer);
        pw.println("123");
        pw.println("485");
        pw.println("67 ");
        assertEquals(writer.toString(), b.toString());
        pw.close();
    }

    // TODO: Add tests for the remaining Board methods and for solve.
}
```

Now open `src/Board.java` and implement `toString()`. Replace its TODO exception with code that *returns* three rows of text, in the format specified previously. However, display "0" as a space instead, and end each row, including the last, with `System.lineSeparator()`. Do not print from this method.



### src/Board.java

```
import java.util.List;

class Board implements Ilayout, Cloneable {
    private static final int dim = 3;
    private int board[][];

    public Board() {
        board = new int[dim][dim];
    }

    public Board(String str) throws IllegalStateException {
        if (str.length() != dim * dim)
            throw new IllegalStateException("Invalid arg in Board constructor");
        board = new int[dim][dim];
        int si = 0;
        for (int i = 0; i < dim; i++)
            for (int j = 0; j < dim; j++)
                board[i][j] = Character.getNumericValue(str.charAt(si++));
    }

    public String toString() {
        // TODO: Three rows, 0 as a space, newline after each row.
        throw new UnsupportedOperationException("TODO: toString");
    }

    public boolean equals(Object o) {
        // TODO: Compare tile positions; handle null/other types.
        throw new UnsupportedOperationException("TODO: equals");
    }

    public int hashCode() {
        // TODO: Equal boards must have equal hash codes.
        throw new UnsupportedOperationException("TODO: hashCode");
    }

    // These three stubs make explicit the methods required by Ilayout.
    public List<Ilayout> children() {
        // TODO: Make independent boards for all legal moves.
        throw new UnsupportedOperationException("TODO: children");
    }

    public boolean isGoal(Ilayout l) {
        // TODO: Compare this configuration with l.
        throw new UnsupportedOperationException("TODO: isGoal");
    }

    public double getK() {
        // TODO: Return the cost of one move.
        throw new UnsupportedOperationException("TODO: getK");
    }
}
```

Run `testConstructor` and `testConstructor2` again. Both should now pass. If either fails, compare the expected and actual text, correcting `toString()` before continuing.

### 6.2 Complete the remaining Board methods

Complete the remaining TODOs in `src/Board.java`, developing tests for each method in `PuzzleUnitTests`:

- `equals(Object)`: check whether two boards have the same tile positions.
- `hashCode()`: return a hash code consistent with `equals`; equal boards must have equal hash codes.
- `isGoal(l)`: check whether this board matches the given goal.
- `getK()`: return the cost of one move, `1.0`.
- `children()`: return all configurations reachable in one legal move, as independent boards that leave the original unchanged.

Run your board tests and correct any failures before continuing.

## 7 Implement the search algorithm

Complete `solve` in `src/BestFirst.java` by implementing the search logic discussed previously.

**src/BestFirst.java – part 1 of 2 (supplied code)**

```
import java.util.*;

class BestFirst {
    protected Queue<State> abertos;
    private Map<Ilayout, State> fechados;
    private State actual;
    private Ilayout objective;

    static class State {
        private Ilayout layout;
        private State father;
        private double g;

        public State(Ilayout l, State n) {
            layout = l;
            father = n;
            if (father != null)
                g = father.g + l.getK();
            else g = 0.0;
        }

        public String toString() { return layout.toString(); }
        public double getG() { return g; }
        public int hashCode() { return toString().hashCode(); }

        public boolean equals(Object o) {
            if (o == null) return false;
            if (this.getClass() != o.getClass()) return false;
            State n = (State) o;
            return this.layout.equals(n.layout);
        }
    }
```

A `State` holds a configuration (`layout`), a parent reference (`father`) and the accumulated cost (`g`), returned by `getG()`. This differs from `Board.getK()`, the cost of one move.

**src/BestFirst.java – part 2 of 2**

```
    final private List<State> sucessores(State n) {
        List<State> sucs = new ArrayList<>();
        List<Ilayout> children = n.layout.children();
        for (Ilayout e : children) {
            if (n.father == null || !e.equals(n.father.layout)) {
                State nn = new State(e, n);
                sucs.add(nn);
            }
        }
        return sucs;
    }

    final public Iterator<State> solve(Ilayout s, Ilayout goal) {
        objective = goal;
        abertos = new PriorityQueue<>(10,
            (s1, s2) -> (int) Math.signum(s1.getG() - s2.getG()));
        fechados = new HashMap<>();
        abertos.add(new State(s, null));
        List<State> sucs;
        // TODO: Complete the search using the diagram in the guide.
        throw new UnsupportedOperationException("TODO: solve");
    }
}
```

Replace the TODO exception in `solve(s, goal)` with your implementation, keeping the supplied initialisation. Return an `Iterator<State>` over the shortest path from start to goal, including both endpoints, or `null` if no solution exists. Develop tests for `solve` in `PuzzleUnitTests` to check that the returned path is valid and shortest.

**src/Main.java – complete starter code**

```
import java.util.Iterator;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        BestFirst s = new BestFirst();
        Iterator<BestFirst.State> it = s.solve(new Board(sc.next()),
            new Board(sc.next()));
        if (it == null) System.out.println("no solution found");
        else {
            while (it.hasNext()) {
                BestFirst.State i = it.next();
                System.out.println(i);
                // The problem requires an integer move count.
                if (!it.hasNext()) System.out.println((int) i.getG());
            }
        }
        sc.close();
    }
}
```

The supplied `Main` reads the input and prints the solution. Once all tests pass, compress the `src` folder containing your completed Java files into a ZIP, keeping `src` as the top-level folder inside the archive. Submit this ZIP to Mooshak, **Contest Artificial Intelligence 2026-27, Problem A – Lab Tutorial 1**, by **28 September 2026**.

# A Small 8-Puzzle Instances

Input consists of two lines: the initial board and the goal, each containing the digits 0–8 once. Output lists every board on the shortest path, with a blank line after each board, followed by the move count as an integer. Spaces represent empty cells.

## A.1 Sample 1

**Sample input 1**

```
023145678
123405678
```

**Sample output 1**

```
 23
145
678

123
 45
678

123
4 5
678

2
```

## A.2 Sample 2

**Sample input 2**

```
123456780
436718520
```

**Output, lines 1–20**

```
123
456
78

123
456
7 8

123
4 6
758

1 3
426
758

 13
426
758
```

**Continued, lines 21–40**

```
413
 26
758

413
726
 58

413
726
5 8

413
7 6
528

4 3
716
528
```

**Continued, lines 41–53**

```
43
716
528

436
71
528

436
718
52

12
```

# B JUnit Assertions

Here is a list of assertions available in JUnit 5.0. Possible arguments are given in parentheses.

**`assertTrue`**
`(boolean)` Reports an error if the value is false.
`(boolean, String)` Adds the error message to the output.

**`assertFalse`**
`(boolean)` Reports an error if the value is true.
`(boolean, String)` Adds the error message to the output.

**`assertNull`**
`(Object)` Reports an error if the object reference is not null.
`(Object, String)` Adds the error message to the output.

**`assertNotNull`**
`(Object)` Reports an error if the object reference is null.
`(Object, String)` Adds the error message to the output.

**`assertSame`**
`(Object, Object)` Reports an error if the references do not identify the same object.
`(Object, Object, String)` Adds the error message to the output.

**`assertNotSame`**
`(Object, Object)` Reports an error if the references identify the same object.
`(Object, Object, String)` Adds the error message to the output.

**`assertEquals`**
`(Object, Object)` Reports an error if the two values are not equal.
`(Object, Object, String)` Adds the error message to the output.
`(String, String)` Reports the differences if the strings are not equal.
`(String, String, String)` Adds the error message to the output.
`(boolean, boolean)` Reports an error if the two values are not equal.
`(boolean, boolean, String)` Adds the error message to the output.
`(byte, byte)` Reports an error if the two values are not equal.
`(byte, byte, String)` Adds the error message to the output.
`(char, char)` Reports an error if the two values are not equal.
`(char, char, String)` Adds the error message to the output.
`(short, short)` Reports an error if the two values are not equal.
`(short, short, String)` Adds the error message to the output.
`(int, int)` Reports an error if the two values are not equal.
`(int, int, String)` Adds the error message to the output.
`(long, long)` Reports an error if the two values are not equal.
`(long, long, String)` Adds the error message to the output.
`(float, float, float)` Reports an error if the first two values differ by more than the third.
`(float, float, float, String)` Adds the error message to the output.
`(double, double, double)` Reports an error if the first two values differ by more than the third.
`(double, double, double, String)` Adds the error message to the output.

**`assertArrayEquals`**
`(Object[], Object[])` Reports an error if array lengths or corresponding elements differ.
`(Object[], Object[], String)` Adds the error message to the output.

You can also test whether code throws an expected exception or completes within a specified time, using assertions inside a `@Test` method.

For example, the following test checks whether invoking a method throws the expected exception:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertThrows;

public class ExceptionExampleTest {
    @Test
    public void testNumberFormatException() {
        assertThrows(NumberFormatException.class, () -> {
            Integer I = Integer.valueOf("not a number");
        });
    }
}
```

You can also check that code takes no more than a specified time to execute. For example, the following test checks that the call to solve completes within one second:

```
    @Test
    public void testEficientSolve() {
        assertTimeoutPreemptively(Duration.ofSeconds(1), () -> {
            BestFirst s = new BestFirst();
            Iterator<BestFirst.State> it = s.solve(new Board("023145678"),
                new Board("123405678"));
        });
    }
```

Learn more about JUnit at junit.org.